

# FASE 2 — Auditoría de Código

## Archivos Analizados

- AuthService.java
- AuthController.java
- UserRepository.java

## AuthService.java

| # | Descripción del problema                                        | Archivo          | Línea aprox.  | Principio violado                  | Riesgo |
|---|-----------------------------------------------------------------|------------------|---------------|------------------------------------|--------|
| 1 | Uso de MD5 para hash de contraseña                              | AuthService.java | 22            | Seguridad - Hashing débil          | Alto   |
| 2 | Retorna el hash de contraseña en la respuesta                   | AuthService.java | 29 / 34       | Seguridad - Exposición de datos    | Alto   |
| 3 | Uso de <code>System.out.println</code> con datos sensibles      | AuthService.java | 25–26 / 31–32 | Seguridad / Clean Code             | Medio  |
| 4 | <code>throws Exception</code> genérico                          | AuthService.java | 20            | Clean Code - Manejo de excepciones | Medio  |
| 5 | Método <code>login</code> concentra múltiples responsabilidades | AuthService.java | 20–35         | SOLID - SRP                        | Medio  |

### Análisis breve:

Se identifican vulnerabilidades críticas en el manejo de contraseñas y exposición de información. Además, el diseño viola SRP y carece de manejo adecuado de errores.

## AuthController.java

| # | Descripción del problema                                          | Archivo             | Línea aprox. | Principio violado             | Riesgo |
|---|-------------------------------------------------------------------|---------------------|--------------|-------------------------------|--------|
| 6 | Uso de <code>@RequestParam</code> para credenciales               | AuthController.java | 21 / 27      | Seguridad - Exposición en URL | Alto   |
| 7 | Retorno de <code>Map&lt;String, Object&gt;</code> en lugar de DTO | AuthController.java | 21 / 27      | Clean Code / SOLID            | Medio  |
| 8 | <code>throws Exception</code> genérico                            | AuthController.java | 21 / 27      | Clean Code                    | Medio  |

| #  | Descripción del problema                | Archivo             | Línea aprox. | Principio violado | Riesgo |
|----|-----------------------------------------|---------------------|--------------|-------------------|--------|
| 9  | Falta de validación de datos de entrada | AuthController.java | 21–29        | Seguridad         | Alto   |
| 10 | Siempre retorna HTTP 200                | AuthController.java | 21–29        | Diseño REST       | Medio  |

**Análisis breve:**  
El controlador expone credenciales por parámetros, no valida entradas y no utiliza códigos HTTP adecuados. Esto afecta seguridad y diseño REST.

## UserRepository.java

| #  | Descripción del problema                                               | Archivo             | Línea aprox.   | Principio violado         | Riesgo |
|----|------------------------------------------------------------------------|---------------------|----------------|---------------------------|--------|
| 11 | Concatenación directa en consultas SQL                                 | UserRepository.java | 19 / 33        | Seguridad - SQL Injection | Alto   |
| 12 | Uso de <code>Statement</code> en vez de <code>PreparedStatement</code> | UserRepository.java | 17 / 31        | Seguridad                 | Alto   |
| 13 | Credenciales de BD hardcodeadas                                        | UserRepository.java | 12–14          | Seguridad                 | Alto   |
| 14 | No se cierran conexiones ni recursos                                   | UserRepository.java | 16–27 / 29–35  | Clean Code                | Alto   |
| 15 | Dependencia directa de JDBC (sin abstracción)                          | UserRepository.java | Clase completa | SOLID - DIP               | Medio  |

**Análisis breve:**  
Se detectan vulnerabilidades críticas como SQL Injection y credenciales expuestas en código. Además, hay mala gestión de recursos y violación del principio de inversión de dependencias.

## Conclusión General Fase 2

El sistema presenta vulnerabilidades críticas de seguridad (SQL Injection, hashing débil, exposición de datos), además de múltiples violaciones a Clean Code y principios SOLID (SRP y DIP). La arquitectura actual requiere refactorización prioritaria para ser viable en un entorno productivo.